

COMP.SE.140

# Project End Report

Anh Pham ([anh.pham@tuni.fi](mailto:anh.pham@tuni.fi))

50357871

# 1. Instructions for teaching assistants

## 1.1. Optional features

- A static analysis pipeline is used for server 1 code and the test scripts (both written in Python). The tool used is *pylint*.
- A monitoring and logging for troubleshooting API is implemented under */metrics*. The metrics include:
  - o Start time of the service
  - o Uptime (time from start to current)
  - o Total number of state change requests sent
  - o Number of successful state change requests (unchanged or invalid state changes won't be counted)

Note: If the application is run as a whole using Docker Compose, the tests will also be triggered, and therefore, the total number of request and number of successful requests will both start at 16 (the number of requests sent by the tests).

## 1.2. Instructions for examiners

### Notes on commits for test-driven development

As instructed in the project description, the tests are written before implementation (the commit pairs for writing tests and its implementation are (f19ef9f9-c64a57f9, bf7ce25f-faeff5fc, 97c02c50-bf8fccd, a0fb320d-9454f916, f1986599-6ec684c3, 8c2256ba-d3729e5b)). There was an issue with running the test pipeline in Gitlab CI that I can only find out the issue at the very end of the project (for some reason the Gitlab CI cannot download the Golang Docker image properly, so I changed the version and it worked). At the time, locally, the tests all failed before the implementation and passed after the implementation (as expected).

### Platform information:

- CPU architecture: amd64
- Memory: 8GB LPDDR4X
- Processor: 1.1GHz Intel Core i5
- Operating system: MacOS 15.2
- Docker: v27.4.1, build b9d17eaebb
- Docker Compose: v1.29.2

### Testing the application

- Clone the application at the *project* branch:

```
git clone -b project git@compse140.devops-  
gitlab.rd.tuni.fi:skanph/remote\_devops\_compose.git
```

- Run the application in Docker:

```
docker-compose up --build
```

- Run test:

```
docker-compose run test
```

- Access the application:  
<http://localhost:8198>  
`curl localhost:8198 -u username:password`
- Requesting state change (credentials required for INIT):  
`curl -X PUT localhost:8198/state -d "RUNNING" -H "Content-Type: text/plain" -H "Accept: text/plain"`
- Getting state:  
`curl localhost:8198/state`
- Request disk information:  
`curl localhost:8198/request`
- Get state changes:  
`curl localhost:8198/run-log`
- Testing optional features:
  - o The *pylint* pipeline is defined in `gitlab-ci.yml`
  - o Logs for troubleshooting:  
`curl localhost:8198/metrics`

## 2. Description of the CI/CD pipeline

### 2.1. Version management

As instructed in the project description, the application is hosted in the *project* branch. The application is also moved to Gitlab to enable Gitlab's CI/CD pipeline:

[https://compse140.devops-gitlab.rd.tuni.fi/skanph/remote\\_devops\\_compose/-/tree/project](https://compse140.devops-gitlab.rd.tuni.fi/skanph/remote_devops_compose/-/tree/project)

### 2.2. Building tools

For local development, the application uses Python for service1 and Golang for service2. Nginx is used as the gateway for load-balancing and access control. Each of the above services is assembled into a Docker image and run together using Docker Compose.

### 2.3. Testing: tools and test cases

- The application uses Python's *unittest* library for testing the application. Docker Compose is also used for test orchestration. The tests files are located in the *tests/* folder.
- Test files:
  - o *test\_exercise4.py*: Initial tests for the front-facing features implemented in exercise4. It's created before any features required by the project instructions in order to test-run the tests in Docker Compose.
  - o *test\_gateway.py*: Ensuring the functionality of state changes and getting run log.
  - o *test\_metrics.py*: Testing endpoint for monitoring and state management.

## 2.4. Packing

All services (service1, service2, nginx, test) are containerized using Docker and Docker Compose to ensure portability and the application can be run and tested in different environments and platforms.

## 2.5. Deployment

All services are deployed using Docker Compose:

```
docker-compose up -build -d
```

Successful deployment can be confirmed by listing the running containers:

```
docker-compose ps
```

## 2.6. Operating and monitoring

- For monitoring, system metrics such as start time, uptime, number of requests, and current state of the application will be showed when a request is sent to `/metrics`. The logs of each service can be retrieved using `docker-compose logs`.
- For operation, a web interface <http://localhost:8198> can be accessed. It provides user-friendly UI with buttons and text fields to request data, change states, get run log, and retrieve metrics.

# 3. Example runs of the pipeline

## 3.1. Successful pipeline

Search visible log output

381 Container remote\_devops\_compose-nginx-1 Creating

382 Container remote\_devops\_compose-nginx-1 Created

383 Container remote\_devops\_compose-test-1 Creating

384 Container remote\_devops\_compose-test-1 Created

385 Container remote\_devops\_compose-service2-1 Starting

386 Container remote\_devops\_compose-service2-1 Started

387 Container remote\_devops\_compose-service1-1 Starting

388 Container remote\_devops\_compose-service1-1 Started

389 Container remote\_devops\_compose-service1-2 Starting

390 Container remote\_devops\_compose-service1-2 Started

391 Container remote\_devops\_compose-service1-3 Starting

392 Container remote\_devops\_compose-service1-3 Started

393 Container remote\_devops\_compose-nginx-1 Starting

394 Container remote\_devops\_compose-nginx-1 Started

395 Container remote\_devops\_compose-test-1 Starting

396 Container remote\_devops\_compose-test-1 Started

397 \$ docker-compose run test || exit 1

398 Container remote\_devops\_compose-service2-1 Running

399 Container remote\_devops\_compose-service1-1 Running

400 Container remote\_devops\_compose-service1-2 Running

401 Container remote\_devops\_compose-service1-3 Running

402 Container remote\_devops\_compose-nginx-1 Running

403 .....

404 -----

405 Ran 6 tests in 5.049s

406 OK

407 Running after\_script

408 Running after script...

409 \$ docker-compose down

410 Container remote\_devops\_compose-test-1 Stopping

411 Container remote\_devops\_compose-test-1 Stopped

412 Container remote\_devops\_compose-test-1 Removing

413 Container remote\_devops\_compose-test-1 Removed

Duration: 2 minutes 50 seconds

Finished: in 5 minutes

Queued: 0 seconds

Timeout: 1h (from project)

Runner: #156 (y-3\_PqNi)

Commit e90723d0

fix test

Pipeline #2698 Passed for project

test

Related jobs

→ test

## 3.2. Failing pipeline

Search visible log output

443 Container remote\_devops\_compose-service1-1 Starting

444 Container remote\_devops\_compose-service1-1 Started

445 Container remote\_devops\_compose-nginx-1 Starting

446 Container remote\_devops\_compose-nginx-1 Started

447 Container remote\_devops\_compose-test-1 Starting

448 Container remote\_devops\_compose-test-1 Started

449 \$ docker-compose run test || exit 1

450 Container remote\_devops\_compose-service2-1 Running

451 Container remote\_devops\_compose-service1-1 Running

452 Container remote\_devops\_compose-service1-2 Running

453 Container remote\_devops\_compose-service1-3 Running

454 Container remote\_devops\_compose-nginx-1 Running

455 ..F...

456 =====

457 FAIL: test\_get\_page (test\_gateway.TestGateway.test\_get\_page)

458 Test that the main page is accessible.

459 -----

460 Traceback (most recent call last):

461 File "/app/tests/test\_gateway.py", line 26, in test\_get\_page

462 self.assertEqual(response.status\_code, 201)

463 AssertionError: 200 != 201

464 -----

465 Ran 6 tests in 5.056s

466 FAILED (failures=1)

467 Running after\_script

468 Running after script...

469 \$ docker-compose down

470 Container remote\_devops\_compose-test-1 Stopping

471 Container remote\_devops\_compose-test-1 Stopped

472 Container remote\_devops\_compose-test-1 Removing

473 Container remote\_devops\_compose-test-1 Removed

Duration: 4 minutes 11 seconds

Finished: 4 minutes ago

Queued: 0 seconds

Timeout: 1h (from project) ?

Runner: #156 (y-3\_PqNi)

Commit a1a83800 

failed test for documentation

Pipeline #2695  Failed for project 

test

Related jobs

→  test

## 4. Reflections

### 4.1. Learnings

- Setting up Gitlab CI/CD pipeline is quite interesting as it is relatively different from those in Github.
- Integrating other tools (pylint) to the pipeline
- TDD. Despite having learnt this development method from other courses, this is the first time I actually got hands on experience with it.
- Some commands on getting metrics of the operating system
- UI development, including JavaScript.
- Docker containers and images.

### 4.2. Difficulties

- Setting up CI/CD is quite a pain (for me as an Intel Mac user). It took me around 9-10 hours in total to properly set up and get the pipeline working after a commit push.
- There is a small but costly problem of a certain Golang Docker image that the Gitlab CI cannot download properly (mentioned at the beginning of Chapter 1.2). I can only apply TDD locally and it does not reflect in the CI pipeline.
- My laptop is quite old so it cannot run the application and the gitlab-runner in Docker and checking the pipeline status in the browser concurrently (especially with the difficulty I mentioned previously as I need to push continuously).
- I cannot find a suitable Golang library for linting that is suitable for Gitlab CI. Therefore, there is no static analysis for service2.

- The problem of the Gitlab server in January caused a delay of my finishing time.

### 4.3. Effort analysis

- Research and planning: 10 hours
- Setting up CI/CD pipeline: 10 hours
- Backend development and Testing: 25 hours
- Frontend development: 10 hours
- Optional features (monitoring and logging): 10 hours
- Documentation: 5 hours

**Total: 70 hours**